

```
# ■ Gini Impurity: Manual + sklearn ■
import numpy as np
from sklearn.tree import DecisionTreeClassifier
import matplotlib.pyplot as plt

# ■ Part 1: Manual Gini calculation ■
def gini_impurity(labels):
    """Calculate Gini impurity for a list of class labels."""
    labels = np.array(labels)
    n = len(labels)
    if n == 0: return 0
    classes = np.unique(labels)
    gini = 1.0
    for c in classes:
        p = np.sum(labels == c) / n # proportion of class c
        gini -= p ** 2
    return gini
```

```
# Test different node compositions
print('Pure node (all same) :', gini_impurity([1,1,1,1,1])) # 0.00
print('50-50 split :', gini_impurity([0,0,0,1,1,1])) # 0.50
print('70-30 split :', gini_impurity([0,0,0,0,0,0,0,1,1,1])) # 0.42
print('90-10 split :', gini_impurity([0]*9 + [1])) # 0.18
# ■ Part 2: Weighted Gini for a split ■
def weighted_gini(left_labels, right_labels):
    """Gini after a split - weighted by group size."""
    n_total = len(left_labels) + len(right_labels)
    w_left = len(left_labels) / n_total
    w_right = len(right_labels) / n_total
    return w_left * gini_impurity(left_labels) + w_right * gini_impurity(right_labels)
    # Two possible splits on a feature - which is better?
# Split A: Glucose <= 120 vs Glucose > 120
left_A = [0,0,0,0,1] # Glucose <= 120: 4 healthy, 1 diabetic
right_A = [0,1,1,1,1] # Glucose > 120: 1 healthy, 4 diabetic
# Split B: Age <= 30 vs Age > 30
left_B = [0,0,1,1,0] # Age <= 30: mixed
right_B = [0,1,0,1,1] # Age > 30: mixed
print(f'Weighted Gini - Glucose split: {weighted_gini(left_A, right_A):.3f}')
print(f'Weighted Gini - Age split : {weighted_gini(left_B, right_B):.3f}')
print('Tree picks the split with LOWER weighted Gini → Glucose split is better!')
# ■ Part 3: See sklearn report the Gini per node ■
from sklearn.datasets import make_classification
from sklearn.tree import export_text
X, y = make_classification(n_samples=200, n_features=4, random_state=42)
tree = DecisionTreeClassifier(max_depth=2, random_state=42)
tree.fit(X, y)
# export_text shows 'gini' value at each node
print(export_text(tree, feature_names=['F1', 'F2', 'F3', 'F4']))
# Each node shows: feature threshold | gini | samples | distribution
```

```
Pure node (all same) : 0.0
50-50 split : 0.5
70-30 split : 0.42000000000000004
90-10 split : 0.17999999999999994
Weighted Gini - Glucose split: 0.320
Weighted Gini - Age split : 0.480
Tree picks the split with LOWER weighted Gini → Glucose split is better!
|--- F1 <= -0.15
| |--- F3 <= -0.44
| | |--- class: 0
| |--- F3 > -0.44
| | |--- class: 0
|--- F1 > -0.15
| |--- F3 <= 0.63
| | |--- class: 1
| |--- F3 > 0.63
| | |--- class: 0
```

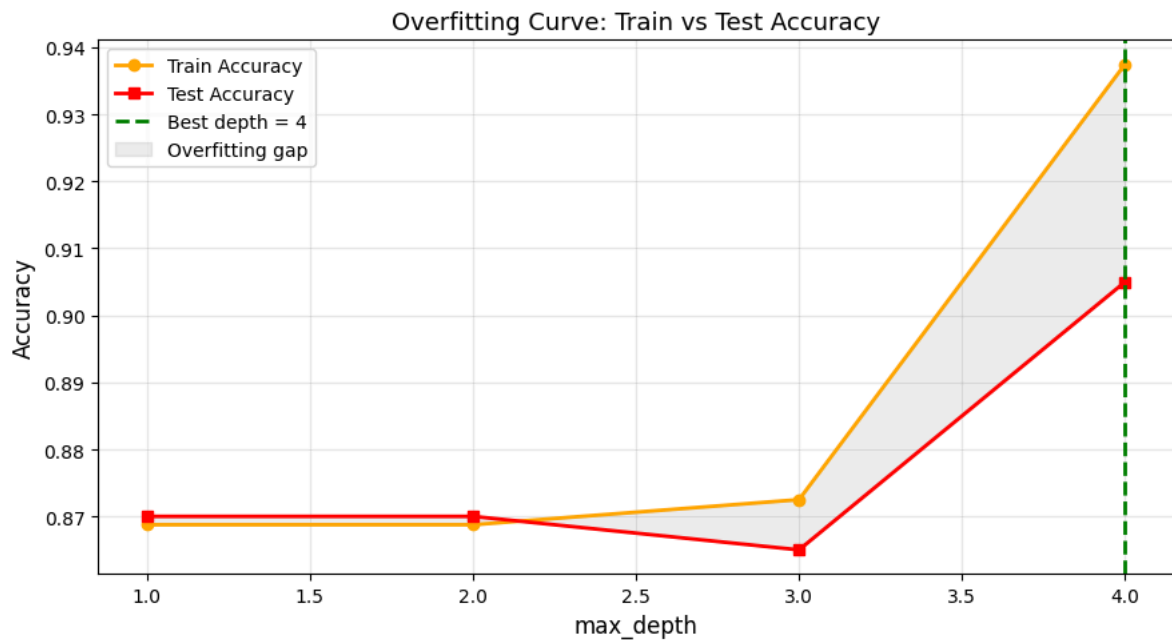
```
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
X, y = make_classification(n_samples=1000, n_features=5, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
depths = range(1, 5)
train_acc, test_acc = [], []
for d in depths:
    tree = DecisionTreeClassifier(max_depth=d, random_state=42)
```

```

tree.fit(X_train, y_train)
train_acc.append(accuracy_score(y_train, tree.predict(X_train)))
test_acc.append(accuracy_score(y_test, tree.predict(X_test)))
best_depth = depths[test_acc.index(max(test_acc))]
plt.figure(figsize=(9, 5))
plt.plot(depths, train_acc, 'o-', color='orange', lw=2, label='Train Accuracy')
plt.plot(depths, test_acc, 's-', color='red', lw=2, label='Test Accuracy')
plt.axvline(x=best_depth, color='green', linestyle='--', lw=2,
            label=f'Best depth = {best_depth}')
plt.fill_between(depths,
                train_acc, test_acc,
                alpha=0.15, color='gray', label='Overfitting gap')
plt.xlabel('max_depth', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.title('Overfitting Curve: Train vs Test Accuracy', fontsize=13)
plt.legend(); plt.grid(alpha=0.3); plt.tight_layout(); plt.show()

# INSIGHT: Train acc always increases → tree memorises more
# Test acc peaks then falls → past the peak = overfitting
print(f'Best test accuracy {max(test_acc):.2%} at depth={best_depth}')

```



Best test accuracy 90.50% at depth=4

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
import numpy as np

configs = [
    {'max_depth': None},
    {'max_depth': 5},
    {'max_depth': 5, 'min_samples_split': 10},
    {'max_depth': 5, 'min_samples_leaf': 5},
    {'max_depth': 5, 'max_features': 'sqrt'}, # Fixed here!
]

print(f'Config:45s {CV Score (mean±std):20s}')
print('-' * 68)
for cfg in configs:
    tree=DecisionTreeClassifier(**cfg, random_state=42)
    scores=cross_val_score(tree,X,y,cv=5,scoring='accuracy')
    print(f'{str(cfg):45s}{scores.mean():.3f} ± {scores.std():.3f}')

```

Config	CV Score (mean±std)
{'max_depth': None}	0.880 ± 0.036
{'max_depth': 5}	0.889 ± 0.024
{'max_depth': 5, 'min_samples_split': 10}	0.890 ± 0.026
{'max_depth': 5, 'min_samples_leaf': 5}	0.898 ± 0.021
{'max_depth': 5, 'max_features': 'sqrt'}	0.894 ± 0.026

Start coding or [generate](#) with AI.

